

Aproximación de una cuerda por infinitos péndulos de longitud infinitesimal

Cesc Barrera y Ernest J. Redolat

Abril 2026

Índice

1. Introducción y Modelización del Sistema	3
1.1. Hipótesis y Aproximaciones	3
2. Formalismo Lagrangiano del Sistema Discreto	4
2.1. Análisis de la Energía Potencial (U)	4
2.2. Análisis de la Energía Cinética (K)	4
2.3. Lagrangiana del Sistema	5
3. El Límite Continuo: Transición a la Mecánica de Medios Continuos	6
3.1. Límite de la Energía Potencial (U)	6
3.2. Límite de la Energía Cinética (K)	6
3.3. Densidad Lagrangiana y Ecuación de Movimiento	6
4. Deducción de la Ecuación de Movimiento: Principio de Mínima Acción	7
4.1. Variación de la Acción y Desarrollo de Taylor	7
4.2. Desglose de la integración por partes	7
4.3. Ecuación de Euler-Lagrange para el medio continuo	8
5. Resolución de la EDP: Separación de Variables y Funciones de Bessel	9
5.1. Resolución de la componente temporal	9
5.2. Transformación a la Ecuación de Bessel	9
5.3. Condiciones de Contorno y Modos Normales	10
6. Análisis de la Solución y Modos Normales	10
6.1. Los ceros de la función de Bessel y frecuencias naturales	10
6.2. Ecuación de movimiento completa	11
7. Ortogonalidad de las soluciones y determinación de los coeficientes	12
7.1. Propiedad de ortogonalidad	12
7.2. Cambio de variable y proyección	12
7.3. Cálculo de A_n	12
7.4. Solución general con coeficientes determinados	13
8. Implementación Práctica: Interpolación y Diferenciación Numérica	14
8.1. Interpolación mediante Polinomios de Lagrange	14
8.2. Resolución Analítica mediante Relaciones de Recurrencia	14
8.3. Enfoque Computacional: Cuadraturas Numéricas	15
9. Métodos numéricos. Simulaciones y análisis	16
9.1. Simulación del sistema discreto lineal	16
9.1.1. Matriz de rigidez y matriz de masas	16
9.1.2. Resolución del sistema de EDOs. Método Runge-Kutta.	17
9.1.3. Modos normales	17
9.2. Simulación del sistema discreto no lineal	18
9.3. Simulación del sistema continuo	19
9.4. Error numérico. Sensibilidad a CI.	20
9.4.1. Péndulo doble. Espacio de fases y sección de Poincaré.	21
10. Análisis del sistema discreto no lineal	23
10.1. Puntos de Equilibrio (PEQs)	23
10.2. Estabilidad	23
A. Código 1: Simulación discreta para ángulos pequeños (sistema lineal)	25
B. Código 2: Simulación discreta (sistema no lineal)	26
C. Código 3: Simulación continua (funciones de Bessel)	28
D. Código 4: Animación de los modos normales (sistema lineal)	31
E. Código 5: Espacio de fases 3D del sistema no lineal	32
F. Código 6: Secciones de Poincaré múltiples para distintos niveles de energía	34

1. Introducción y Modelización del Sistema

La transición entre la mecánica de sistemas discretos y la mecánica de medios continuos constituye uno de los pilares sobre los que se asienta la física teórica moderna. El estudio de una cuerda bajo la acción de la gravedad, aunque aparentemente trivial, representa un desafío analítico que obliga a conectar el formalismo lagrangiano de n partículas con la teoría de campos y las funciones especiales.

Históricamente, este problema hunde sus raíces en el siglo XVIII. Fue Daniel Bernoulli quien, en 1732, propuso modelizar una cuerda colgante como una sucesión finita de péndulos acoplados para deducir sus modos normales de vibración. Este enfoque no solo permitió comprender la dinámica de las cuerdas vibrantes, sino que sentó las bases de lo que más tarde serían las series de Fourier y la resolución de ecuaciones diferenciales mediante funciones ortogonales.

En el presente trabajo, nos proponemos derivar la ecuación de movimiento de una cuerda ideal de densidad lineal uniforme ($\lambda = M/L$) y longitud L , suspendida de un extremo fijo. Para alcanzar una solución de alto rigor matemático, el desarrollo se estructurará en dos etapas críticas:

1. **Modelización Discreta:** La cuerda se aproximará inicialmente como un sistema de n masas puntuales unidas por segmentos rígidos sin masa. Utilizaremos el formalismo de Euler-Lagrange para describir las interacciones y la transferencia de tensión a lo largo de la cadena.
2. **Límite Continuo:** Mediante el paso al límite cuando el número de péndulos $n \rightarrow \infty$ y la masa de cada elemento $m \rightarrow 0$, transformaremos el conjunto de ecuaciones diferenciales ordinarias acopladas en una ecuación en derivadas parciales que gobierna el desplazamiento lateral de la cuerda.

1.1. Hipótesis y Aproximaciones

Para garantizar la solubilidad analítica del sistema sin perder la esencia física del fenómeno, se asumen las siguientes condiciones:

- **Régimen de pequeñas oscilaciones:** Se considera que el ángulo de deflexión θ es lo suficientemente pequeño como para validar las aproximaciones $\sin \theta \approx \theta$ y $\cos \theta \approx 1 - \frac{1}{2}\theta^2$.
- **Cuerda ideal:** Se desprecia la rigidez a la flexión, los efectos disipativos del aire y cualquier deformación elástica longitudinal.
- **Tensión gravitatoria:** A diferencia de una cuerda en tensión constante, la fuerza recuperadora en este sistema dependerá intrínsecamente de la altura, siendo nula en el extremo libre y máxima en el punto de suspensión.

2. Formalismo Lagrangiano del Sistema Discreto

Para abordar la dinámica de la cuerda, comenzamos modelizándola como una cadena de n péndulos acoplados. Consideramos que la cuerda tiene una masa total M y una longitud L , por lo que definimos para cada elemento:

- $l = \frac{L}{n}$: longitud de cada segmento rígido.
- $m = \frac{M}{n} = \lambda l$: masa de cada partícula puntual situada al final de cada segmento.
- $\theta_i(t)$: ángulo del segmento i -ésimo respecto a la vertical.

2.1. Análisis de la Energía Potencial (U)

La energía potencial del sistema es la suma de las energías potenciales gravitatorias de cada una de las n masas. Tomando el punto de suspensión como origen de coordenadas ($y = 0$), la posición vertical de la masa k -ésima viene dada por la suma de las proyecciones verticales de los segmentos que tiene por encima:

$$y_k = -l \sum_{i=1}^k \cos \theta_i \quad (1)$$

La energía potencial total U se define como:

$$U = \sum_{k=1}^n mgy_k = -mgl \sum_{k=1}^n \sum_{i=1}^k \cos \theta_i \quad (2)$$

Para simplificar esta expresión, realizamos un reordenamiento del sumatorio doble (sumación por partes discreta). Observamos que el ángulo θ_1 aparece en la expresión de la altura de todas las masas (n veces), el ángulo θ_2 aparece en $n - 1$ masas, y así sucesivamente. Por lo tanto:

$$U = -mgl \sum_{i=1}^n (n - i + 1) \cos \theta_i \quad (3)$$

Considerando el régimen de *pequeñas oscilaciones*, aplicamos el desarrollo de Taylor $\cos \theta_i \approx 1 - \frac{1}{2}\theta_i^2$. Despreciando los términos constantes (que no afectan a las ecuaciones del movimiento), el potencial se reduce a:

$$U \approx \frac{1}{2}mgl \sum_{i=1}^n (n - i + 1) \theta_i^2 \quad (4)$$

2.2. Análisis de la Energía Cinética (K)

En el régimen de pequeñas oscilaciones, el movimiento vertical es de segundo orden respecto al horizontal ($\dot{y}_k \approx 0$), por lo que la energía cinética reside fundamentalmente en la velocidad horizontal. La posición horizontal x_k de la masa k es:

$$x_k = l \sum_{i=1}^k \sin \theta_i \approx l \sum_{i=1}^k \theta_i \quad (5)$$

Derivando respecto al tiempo, obtenemos la velocidad de la partícula k :

$$v_{x,k} = \dot{x}_k = l \sum_{i=1}^k \dot{\theta}_i \quad (6)$$

La energía cinética total del sistema es la suma de las energías de cada partícula:

$$K = \frac{1}{2}m \sum_{k=1}^n v_{x,k}^2 = \frac{1}{2}ml^2 \sum_{k=1}^n \left(\sum_{i=1}^k \dot{\theta}_i \right)^2 \quad (7)$$

2.3. Lagrangiana del Sistema

Combinando los resultados anteriores, definimos la función Lagrangiana $\mathcal{L} = K - U$ como:

$$\mathcal{L} = \frac{1}{2}ml^2 \sum_{k=1}^n \left(\sum_{i=1}^k \dot{\theta}_i \right)^2 - \frac{1}{2}mgl \sum_{i=1}^n (n - i + 1)\theta_i^2 \quad (8)$$

Esta expresión representa la dinámica completa del sistema discreto y servirá como base fundamental para realizar el paso al límite continuo.

3. El Límite Continuo: Transición a la Mecánica de Medios Continuos

Para realizar la transición al continuo, tomamos el límite cuando el número de segmentos $n \rightarrow \infty$ y la longitud de cada uno $l \rightarrow 0$, manteniendo la longitud total $L = nl$ constante. En este proceso, la posición discreta de la masa i -ésima se convierte en una variable continua $y \in [0, L]$.

El desplazamiento horizontal de la cuerda en cada punto y y en cada instante t vendrá definido por la función $x(y, t)$. La relación entre el ángulo del segmento y el desplazamiento lateral se obtiene observando la pendiente de la cuerda:

$$\theta_i \approx \sin \theta_i \approx \frac{\Delta x}{\Delta y} \longrightarrow \frac{\partial x(y, t)}{\partial y} \quad (9)$$

3.1. Límite de la Energía Potencial (U)

Partimos de la expresión discreta del potencial para pequeñas oscilaciones:

$$U \approx \frac{1}{2} mgl \sum_{i=1}^n (n - i + 1) \theta_i^2 \quad (10)$$

Sustituyendo $m = \lambda l$ y reconociendo que el término $l(n - i + 1)$ representa la distancia desde el extremo inferior de la cuerda hasta el punto y (es decir, la longitud de cuerda que cuelga, $L - y$), la suma se transforma en la siguiente integral:

$$U = \frac{1}{2} \int_0^L \lambda g(L - y) \left(\frac{\partial x}{\partial y} \right)^2 dy \quad (11)$$

Aquí, identificamos el término $T(y) = \lambda g(L - y)$ como la **tensión local** de la cuerda. Es fundamental notar que la tensión es máxima en el anclaje ($y = 0$, donde $T = \lambda gL = Mg$) y nula en el extremo libre ($y = L$).

3.2. Límite de la Energía Cinética (K)

En el sistema discreto, la velocidad horizontal de la masa k era $v_{x,k} = l \sum_{i=1}^k \dot{\theta}_i$. En el límite continuo, esta acumulación de velocidades angulares se traduce simplemente en la derivada temporal del desplazamiento horizontal:

$$v_x(y, t) = \frac{\partial x(y, t)}{\partial t} \quad (12)$$

Sustituyendo $m = \lambda dy$, la energía cinética total del sistema continuo se expresa como:

$$K = \frac{1}{2} \int_0^L \lambda \left(\frac{\partial x}{\partial t} \right)^2 dy \quad (13)$$

3.3. Densidad Lagrangiana y Ecuación de Movimiento

Definimos la **densidad Lagrangiana** $\mathcal{L}$ tal que la Lagrangiana total es $L_{tot} = \int \mathcal{L} dy$:

$$\mathcal{L} = K_{dens} - U_{dens} = \frac{1}{2} \lambda \left(\frac{\partial x}{\partial t} \right)^2 - \frac{1}{2} \lambda g(L - y) \left(\frac{\partial x}{\partial y} \right)^2 \quad (14)$$

4. Deducción de la Ecuación de Movimiento: Principio de Mínima Acción

Para obtener la ecuación de movimiento de forma rigurosa, partimos del **Principio de Hamilton**. Definimos la acción $\mathcal{S}$ como el funcional que integra la densidad lagrangiana $\mathcal{L}$ en el dominio espacial de la cuerda $[0, L]$ y en el intervalo temporal $[t_1, t_2]$:

$$\mathcal{S}[x(y, t)] = \int_{t_1}^{t_2} \int_0^L \mathcal{L}(x, x_t, x_y, y, t) dy dt \quad (15)$$

4.1. Variación de la Acción y Desarrollo de Taylor

Partimos de la definición de la acción para la trayectoria real $x(y, t)$:

$$\mathcal{S}[x(y, t)] = \int_{t_1}^{t_2} \int_0^L \mathcal{L}(x, x_t, x_y, y, t) dy dt \quad (16)$$

Siguiendo el principio de la mínima acción, introducimos un pequeño desfase o variación infinitesimal, pasando de la trayectoria real x a una configuración perturbada $x + \delta x$. La variación de la acción se define como la diferencia entre la acción de la trayectoria perturbada y la original:

$$\delta\mathcal{S} = \int_{t_1}^{t_2} \int_0^L [\mathcal{L}(x + \delta x, x_t + \delta x_t, x_y + \delta x_y, y, t) - \mathcal{L}(x, x_t, x_y, y, t)] dy dt \quad (17)$$

Dado que el desplazamiento δx es infinitamente pequeño, podemos tratar los términos del integrando mediante un **desarrollo de Taylor multivariable** alrededor de la trayectoria no perturbada. Al ser una variación de primer orden, despreciamos los términos cuadráticos y superiores ($\delta^2 \approx 0$):

$$\mathcal{L}(x + \delta x, x_t + \delta x_t, x_y + \delta x_y) \approx \mathcal{L} + \frac{\partial \mathcal{L}}{\partial x} \delta x + \frac{\partial \mathcal{L}}{\partial x_t} \delta x_t + \frac{\partial \mathcal{L}}{\partial x_y} \delta x_y \quad (18)$$

Sustituyendo este desarrollo en la expresión de $\delta\mathcal{S}$, el término $\mathcal{L}(x, x_t, x_y, y, t)$ se cancela, resultando en la expresión variacional buscada:

$$\delta\mathcal{S} = \int_{t_1}^{t_2} \int_0^L \left[\frac{\partial \mathcal{L}}{\partial x} \delta x + \frac{\partial \mathcal{L}}{\partial x_t} \delta x_t + \frac{\partial \mathcal{L}}{\partial x_y} \delta x_y \right] dy dt = 0 \quad (19)$$

Es importante destacar que en este proceso las variables independientes (y, t) mantienen su valor constante, ya que el desplazamiento es puramente funcional en la coordenada x .

4.2. Desglose de la integración por partes

Analizamos los términos de derivadas por separado:

1. **Término temporal:** Aplicando la regla del producto para la derivada temporal:

$$\int_{t_1}^{t_2} \frac{\partial \mathcal{L}}{\partial x_t} \frac{\partial(\delta x)}{\partial t} dt = \left[\frac{\partial \mathcal{L}}{\partial x_t} \delta x \right]_{t_1}^{t_2} - \int_{t_1}^{t_2} \frac{\partial}{\partial t} \left(\frac{\partial \mathcal{L}}{\partial x_t} \right) \delta x dt \quad (20)$$

El término de frontera se anula ya que la variación δx es cero en t_1 y t_2 .

2. **Término espacial:** Procediendo de igual manera para la coordenada y :

$$\int_0^L \frac{\partial \mathcal{L}}{\partial x_y} \frac{\partial(\delta x)}{\partial y} dy = \left[\frac{\partial \mathcal{L}}{\partial x_y} \delta x \right]_0^L - \int_0^L \frac{\partial}{\partial y} \left(\frac{\partial \mathcal{L}}{\partial x_y} \right) \delta x dy \quad (21)$$

En $y = 0$, la cuerda está anclada, por lo que $\delta x(0, t) = 0$. En $y = L$, el término $\frac{\partial \mathcal{L}}{\partial x_y} = -\lambda g(L - y)x_y$ se anula debido a que la tensión es nula en el extremo libre ($L - L = 0$).

4.3. Ecuación de Euler-Lagrange para el medio continuo

Sustituyendo estos resultados en la variación de la acción, agrupamos todos los términos que dependen de δx :

$$\delta \mathcal{S} = \int_{t_1}^{t_2} \int_0^L \left[\frac{\partial \mathcal{L}}{\partial x} - \frac{\partial}{\partial t} \left(\frac{\partial \mathcal{L}}{\partial x_t} \right) - \frac{\partial}{\partial y} \left(\frac{\partial \mathcal{L}}{\partial x_y} \right) \right] \delta x dy dt = 0 \quad (22)$$

Por el Lema Fundamental del Cálculo de Variaciones, dado que δx es arbitrario, el integrando debe ser idénticamente nulo:

$$\frac{\partial \mathcal{L}}{\partial x} - \frac{\partial}{\partial t} \left(\frac{\partial \mathcal{L}}{\partial x_t} \right) - \frac{\partial}{\partial y} \left(\frac{\partial \mathcal{L}}{\partial x_y} \right) = 0 \quad (23)$$

Sustituyendo nuestra densidad lagrangiana $\mathcal{L} = \frac{1}{2}\lambda x_t^2 - \frac{1}{2}\lambda g(L - y)x_y^2$:

- $\frac{\partial \mathcal{L}}{\partial x} = 0$ (La energía no depende de la posición absoluta horizontal).
- $\frac{\partial \mathcal{L}}{\partial x_t} = \lambda x_t \implies \frac{\partial}{\partial t}(\dots) = \lambda \frac{\partial^2 x}{\partial t^2}$
- $\frac{\partial \mathcal{L}}{\partial x_y} = -\lambda g(L - y)x_y \implies \frac{\partial}{\partial y}(\dots) = -\frac{\partial}{\partial y} \left[\lambda g(L - y) \frac{\partial x}{\partial y} \right]$

Llegamos finalmente a la ecuación de movimiento:

$$\lambda \frac{\partial^2 x}{\partial t^2} = \frac{\partial}{\partial y} \left[\lambda g(L - y) \frac{\partial x}{\partial y} \right] \quad (24)$$

5. Resolución de la EDP: Separación de Variables y Funciones de Bessel

La ecuación de movimiento obtenida es una ecuación en derivadas parciales (EDP) lineal y estacionaria:

$$\lambda \frac{\partial^2 x}{\partial t^2} = \frac{\partial}{\partial y} \left[\lambda g(L-y) \frac{\partial x}{\partial y} \right] \quad (25)$$

Dada la independencia explícita del tiempo en los coeficientes, proponemos una solución mediante el método de **separación de variables**:

$$x(y, t) = Y(y)T(t) \quad (26)$$

Al sustituir esta expresión en la EDP y dividir por $\lambda Y T$, obtenemos:

$$\frac{1}{T} \frac{d^2 T}{dt^2} = \frac{g}{Y} \frac{d}{dy} \left[(L-y) \frac{dY}{dy} \right] = -\omega^2 \quad (27)$$

Donde $-\omega^2$ es la constante de separación (elegida negativa para representar soluciones oscilatorias).

5.1. Resolución de la componente temporal

Tras la separación de variables, la ecuación que gobierna la evolución temporal del sistema es:

$$\frac{d^2 T(t)}{dt^2} + \omega^2 T(t) = 0 \quad (28)$$

Esta expresión corresponde a la ecuación de un **oscilador armónico simple**. La solución general para esta EDO lineal de segundo orden con coeficientes constantes es una combinación lineal de funciones exponenciales complejas o, equivalentemente, de funciones trigonométricas:

$$T(t) = A \cos(\omega t) + B \sin(\omega t) \quad (29)$$

Para simplificar el análisis y centrarnos en los modos normales, podemos expresar la solución de forma compacta utilizando un desfase ϕ :

$$T(t) = C \cos(\omega t + \phi) \quad (30)$$

Donde:

- ω representa la frecuencia angular de oscilación del modo correspondiente.
- C y ϕ son constantes que dependen de las condiciones iniciales de posición y velocidad de la cuerda.

Dado que estamos modelizando un sistema físico conservativo (sin fricción), esta solución garantiza que cada punto de la cuerda realice un movimiento armónico simple con la misma frecuencia ω , permitiendo que la forma general de la oscilación se mantenga constante en el tiempo.

5.2. Transformación a la Ecuación de Bessel

La parte temporal tiene la solución conocida $T(t) = \cos(\omega t + \phi)$. Para la parte espacial, realizamos el cambio de variable $z = L-y$ (donde z es la distancia desde el extremo libre), lo que implica $d/dy = -d/dz$:

$$z \frac{d^2 Y}{dz^2} + \frac{dY}{dz} + \frac{\omega^2}{g} Y = 0 \quad (31)$$

Para identificar esta EDO como una de tipo Bessel, realizamos un segundo cambio de variable: $u = 2\omega \sqrt{\frac{z}{g}}$. Tras aplicar la regla de la cadena para las derivadas, la ecuación se transforma en la **ecuación de Bessel de orden cero**:

$$u^2 \frac{d^2 Y}{du^2} + u \frac{dY}{du} + u^2 Y = 0 \quad (32)$$

La solución general para esta ecuación es una combinación lineal de las funciones de Bessel de primera y segunda especie:

$$Y(u) = C_1 J_0(u) + C_2 Y_0(u) \quad (33)$$

5.3. Condiciones de Contorno y Modos Normales

Debido a la naturaleza física del problema, el desplazamiento Y debe ser finito en el extremo libre ($z = 0$, lo que implica $u = 0$). Como la función $Y_0(u)$ diverge cuando $u \rightarrow 0$, debemos imponer que $C_2 = 0$. Por lo tanto, la solución espacial es:

$$Y(y) = C_1 J_0 \left(2\omega \sqrt{\frac{L-y}{g}} \right) \quad (34)$$

Finalmente, aplicamos la condición de contorno en el punto de anclaje ($y = 0$), donde el desplazamiento es nulo: $x(0, t) = 0 \Rightarrow Y(0) = 0$. Esto nos da la condición para las frecuencias naturales de la cuerda:

$$J_0 \left(2\omega \sqrt{\frac{L}{g}} \right) = 0 \quad (35)$$

6. Análisis de la Solución y Modos Normales

Una vez establecida la solución espacial $Y(u) = C_1 J_0(u)$, debemos aplicar las condiciones físicas que restringen los valores de la frecuencia angular ω .

6.1. Los ceros de la función de Bessel y frecuencias naturales

La condición de contorno en el punto de anclaje ($y = 0$, lo que equivale a $z = L$) exige que el desplazamiento sea nulo:

$$Y(y=0) = J_0 \left(2\omega \sqrt{\frac{L}{g}} \right) = 0 \quad (36)$$

Para que esta igualdad se cumpla, el argumento de la función debe coincidir con uno de los ceros de la función de Bessel de orden cero. Si denotamos por α_n al n -ésimo cero de J_0 (donde $\alpha_1 \approx 2,4048$, $\alpha_2 \approx 5,5201, \dots$), obtenemos las frecuencias naturales de vibración del sistema:

$$2\omega_n \sqrt{\frac{L}{g}} = \alpha_n \implies \omega_n = \frac{\alpha_n}{2} \sqrt{\frac{g}{L}} \quad (37)$$

Cada valor de ω_n define un **modo normal de vibración**. A diferencia de una cuerda tensa convencional donde las frecuencias son múltiplos enteros (armónicos), aquí las frecuencias dependen de las raíces de J_0 , lo que implica que no son armónicas entre sí.

6.2. Ecuación de movimiento completa

Combinando la solución temporal y la espacial para cada modo n , y aplicando el principio de superposición lineal, la solución general para el desplazamiento horizontal $x(y, t)$ es:

$$x(y, t) = \sum_{n=1}^{\infty} J_0 \left(\alpha_n \sqrt{\frac{L-y}{L}} \right) \left[A_n \cos\left(\frac{\alpha_n}{2} \sqrt{\frac{g}{L}} t + \phi\right) \right] \quad (38)$$

7. Ortogonalidad de las soluciones y determinación de los coeficientes

Para determinar los coeficientes A_n a partir de las condiciones iniciales de la cuerda, utilizamos la propiedad de ortogonalidad de las funciones de Bessel. Consideramos que la configuración inicial de la cuerda en $t = 0$ viene dada por una función $x(y, 0) = f(y)$.

7.1. Propiedad de ortogonalidad

La condición de ortogonalidad para las funciones de Bessel de primer orden en un intervalo unitario se define como:

$$\int_0^1 \xi J_\nu(\alpha_n \xi) J_\nu(\alpha_m \xi) d\xi = \begin{cases} 0 & n \neq m \\ \frac{1}{2} J_{\nu+1}^2(\alpha_n) & n = m \end{cases} \quad (39)$$

7.2. Cambio de variable y proyección

Partimos de la superposición de los modos normales en el instante inicial (asumiendo $\phi = 0$ para simplificar, según la condición de reposo inicial):

$$f(y) = \sum_{n=1}^{\infty} A_n J_0 \left(\alpha_n \sqrt{\frac{L-y}{L}} \right) \quad (40)$$

Para aplicar la propiedad de ortogonalidad, realizamos el cambio de variable $\xi = \sqrt{\frac{L-y}{L}}$. Esto implica:

- $y = L(1 - \xi^2)$
- $dy = -2L\xi d\xi$
- Límites de integración: si $y = 0 \rightarrow \xi = 1$; si $y = L \rightarrow \xi = 0$

Multiplicando ambos lados por $\xi J_0(\alpha_m \xi)$ e integrando en el dominio transformado:

$$\int_0^1 \xi f(L(1 - \xi^2)) J_0(\alpha_m \xi) d\xi = \sum_{n=1}^{\infty} A_n \int_0^1 \xi J_0(\alpha_n \xi) J_0(\alpha_m \xi) d\xi \quad (41)$$

7.3. Cálculo de A_n

Debido a la ortogonalidad, todos los términos del sumatorio donde $n \neq m$ se anulan. Aplicando el resultado para $n = m$:

$$\int_0^1 \xi f(L(1 - \xi^2)) J_0(\alpha_n \xi) d\xi = A_n \frac{1}{2} J_1^2(\alpha_n) \quad (42)$$

Aislado el coeficiente y deshaciendo el cambio de variable para expresarlo en términos de y :

$$A_n = \frac{2}{J_1^2(\alpha_n)} \int_0^1 \xi f(L(1 - \xi^2)) J_0(\alpha_n \xi) d\xi \quad (43)$$

Sustituyendo de nuevo $\xi = \sqrt{\frac{L-y}{L}}$ y $d\xi = \frac{-dy}{2L\sqrt{\frac{L-y}{L}}}$, obtenemos la expresión final para los coeficientes del desarrollo:

$$A_n = \frac{1}{L J_1^2(\alpha_n)} \int_0^L f(y) J_0 \left(\alpha_n \sqrt{\frac{L-y}{L}} \right) dy \quad (44)$$

7.4. Solución general con coeficientes determinados

Una vez calculada la expresión para los coeficientes A_n mediante la propiedad de ortogonalidad, podemos escribir la solución completa para el desplazamiento horizontal $x(y, t)$ de la cuerda. Sustituyendo el valor de A_n obtenido en la ecuación (38) y asumiendo que la cuerda parte del reposo ($\phi_n = 0$), la solución definitiva es:

$$x(y, t) = \sum_{n=1}^{\infty} \left[\frac{1}{L J_1^2(\alpha_n)} \int_0^L f(y') J_0 \left(\alpha_n \sqrt{\frac{L-y'}{L}} \right) dy' \right] J_0 \left(\alpha_n \sqrt{\frac{L-y}{L}} \right) \cos \left(\frac{\alpha_n}{2} \sqrt{\frac{g}{L}} t \right) \quad (45)$$

Donde:

- $f(y')$ representa la configuración o forma inicial de la cuerda en el instante $t = 0$.
- α_n son los ceros de la función de Bessel de orden cero J_0 .
- J_1 es la función de Bessel de primer orden.
- El término entre corchetes representa la amplitud de cada modo normal, determinada por la proyección de la forma inicial sobre la base de funciones ortogonales del sistema.

8. Implementación Práctica: Interpolación y Diferenciación Numérica

En situaciones reales, la configuración inicial de la cuerda $f(y)$ no suele conocerse como una función continua, sino como un conjunto discreto de puntos (y_i, x_i) obtenidos, por ejemplo, mediante el procesamiento de una imagen o sensores de posición.

8.1. Interpolación mediante Polinomios de Lagrange

Para reconstruir una función continua $f(y)$ a partir de $N + 1$ puntos de datos, empleamos la interpolación polinómica de Lagrange. El polinomio de grado N viene dado por:

$$P(y) = \sum_{i=0}^N x_i L_i(y) \quad (46)$$

Donde $L_i(y)$ son los polinomios base de Lagrange:

$$L_i(y) = \prod_{j=0, j \neq i}^N \frac{y - y_j}{y_i - y_j} \quad (47)$$

Es importante notar que, para valores grandes de N , el uso de nodos equiespaciados puede conducir al fenómeno de Runge, donde aparecen errores muy significativos en los extremos del intervalo. Para mitigar esto en un contexto de ingeniería física, se recomienda el uso de nodos de Chebyshev o una aproximación por splines cúbicos si la precisión local es prioritaria.

8.2. Resolución Analítica mediante Relaciones de Recurrencia

Una vez obtenida la aproximación $f(y) \approx P(y)$, el cálculo de los coeficientes A_n requiere resolver:

$$I = \int_0^L P(y) J_0 \left(\alpha_n \sqrt{\frac{L-y}{L}} \right) dy \quad (48)$$

Realizando el cambio de variable $\xi = \sqrt{\frac{L-y}{L}}$, la integral se transforma en:

$$I = 2L \int_0^1 \xi \tilde{P}(\xi) J_0(\alpha_n \xi) d\xi \quad (49)$$

Para resolverla, aplicamos la **relación de recurrencia diferencial**: $\frac{d}{dz}[z^\nu J_\nu(z)] = z^\nu J_{\nu-1}(z)$. Para el caso base ($\nu = 1$), sabemos que $\int z J_0(z) dz = z J_1(z)$. Si el polinomio es de mayor grado (ej. ξ^2), procedemos por partes:

1. Identificamos $u = \xi^2$ y $dv = \xi J_0(\alpha_n \xi) d\xi$.
2. La primitiva es $v = \frac{\xi}{\alpha_n} J_1(\alpha_n \xi)$.
3. Resultando en:

$$\int \xi^3 J_0(\alpha_n \xi) d\xi = \frac{\xi^3}{\alpha_n} J_1(\alpha_n \xi) - \frac{2}{\alpha_n} \int \xi^2 J_1(\alpha_n \xi) d\xi \quad (50)$$

Este proceso reduce el grado del polinomio aumentando el orden de la función de Bessel ($J_1, J_2, \dots$) hasta cerrar la solución.

8.3. Enfoque Computacional: Cuadraturas Numéricas

Desde un enfoque de ingeniería, el método más robusto es evitar el desarrollo analítico y aplicar una Cuadratura de Gauss-Legendre o Clenshaw-Curtis. Esto transforma la integral en una suma pesada sobre los puntos capturados:

$$A_n \approx \frac{1}{LJ_1^2(\alpha_n)} \sum_{k=1}^M w_k f(y_k) J_0 \left(\alpha_n \sqrt{\frac{L - y_k}{L}} \right) \quad (51)$$

donde w_k son los pesos de la cuadratura, permitiendo una resolución eficiente e independiente de la complejidad de la forma inicial $f(y)$.

9. Métodos numéricos. Simulaciones y análisis

Llegados a este punto del trabajo, es lógico indagar en los métodos numéricos necesarios para representar el movimiento tanto del sistema discreto de n péndulos como de la cuerda continua. Para ello, emplearemos el programa de cálculo numérico MATLAB.

9.1. Simulación del sistema discreto lineal

Recuperando la ecuación de la Lagrangiana para el sistema de n péndulos acoplados para ángulos pequeños (Ecuación 8),

$$\mathcal{L} = \frac{1}{2}ml^2 \sum_{k=1}^n \left(\sum_{i=1}^k \dot{\theta}_i \right)^2 - \frac{1}{2}mgl \sum_{i=1}^n (n-i+1)\theta_i^2$$

aplicaremos la ecuación de Euler-Lagrange para obtener el sistema matricial.

Cabe destacar que para una n pequeña, el sistema lineal se puede resolver con lápiz y papel, pero para n mayores es muy tedioso y se requieren métodos numéricos para invertir matrices y resolver el sistema (para $n=3$ hay que encontrar núcleos y raíces de polinomios característicos de matrices 6×6).

9.1.1. Matriz de rigidez y matriz de masas

Expandiendo el término de la derivada espacial de la ecuación $(\frac{\partial \mathcal{L}}{\partial \theta_i})$ generamos la matriz de rigidez. La energía potencial total es (Ecuación 4):

$$U \approx \frac{1}{2}mgl \sum_{i=1}^n (n-i+1)\theta_i^2$$

Como no hay términos cruzados (como $\theta_i\theta_j$), la matriz es puramente diagonal. El coeficiente que acompaña a cada $\frac{1}{2}\theta_i^2$ es $mgl(n-i+1)$. Por tanto:

$$K_m = mgl \begin{pmatrix} n & 0 & \dots & 0 \\ 0 & n-1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}$$

Por otro lado, expandiendo el término de la derivada temporal de la ecuación $(\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{\theta}_i})$ obtenemos la matriz de masas (refleja la inercia acoplada del sistema). La energía cinética total es (Ecuación 7):

$$K = \frac{1}{2}m \sum_{k=1}^n v_k^2 = \frac{1}{2}ml^2 \sum_{k=1}^n \left(\sum_{i=1}^k \dot{\theta}_i \right)^2$$

Al expandir el cuadrado del sumatorio interno, los términos $\dot{\theta}_i\dot{\theta}_j$ aparecen múltiples veces. El elemento M_{ij} de la matriz de masa representa cuántas veces sobrevive el producto $\dot{\theta}_i\dot{\theta}_j$ en el sumatorio total. Esto ocurre para todos los índices k que sean mayores o iguales al máximo entre i y j . Por lo tanto, $M_{ij} = ml^2(n - \max(i, j) + 1)$, resultando en:

$$M = ml^2 \begin{pmatrix} n & n-1 & n-2 & \dots & 1 \\ n-1 & n-1 & n-2 & \dots & 1 \\ n-2 & n-2 & n-2 & \dots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \dots & 1 \end{pmatrix}$$

Así pues, la ecuación Euler-Lagrange resulta en la siguiente expresión: $M\ddot{\theta} + K_m\theta = 0$, donde $\ddot{\theta}$ es el vector de aceleraciones angulares y θ el vector de ángulos.

9.1.2. Resolución del sistema de EDOs. Método Runge-Kutta.

Para que MATLAB pueda integrar el movimiento, es necesario primero despejar la aceleración tal que $\ddot{\theta} = -M^{-1}K\theta$ para cada instante de tiempo, es decir, resolver el sistema lineal. De este modo, obtenemos el vector $\ddot{\theta}(\theta(t))$.

Posteriormente, definimos una función para el estado del sistema $y = [\theta; \dot{\theta}]$, reduciendo el sistema de 2º orden a uno de primer orden duplicando el número de ecuaciones mediante el cambio de variable $\omega_i = \dot{\theta}_i$ y $\dot{\omega}_i = \ddot{\theta}_i$.

$$\begin{bmatrix} \dot{\theta}_1 \\ \vdots \\ \dot{\theta}_n \\ \dot{\omega}_1 \\ \vdots \\ \dot{\omega}_n \end{bmatrix} = \begin{bmatrix} \mathbf{0}_{n \times n} & \mathbf{I}_{n \times n} \\ -\mathbf{M}^{-1}\mathbf{K} & \mathbf{0}_{n \times n} \end{bmatrix} \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \\ \omega_1 \\ \vdots \\ \omega_n \end{bmatrix}$$

Finalmente, usamos el método de Runge-Kutta (que es en el que se basa ode45) para resolver las EDOs. Este algoritmo nos devuelve una matriz que tiene por filas la evolución del estado del sistema (posiciones y velocidades), es decir, el estado en cada diferencial de tiempo del intervalo configurado. De aquí extraemos la matriz de la evolución de las posiciones.

Para realizar la animación, sólo hace falta crear una gráfica animada actualizando la posición de cada masa en cada diferencial de tiempo a partir del ángulo mediante las relaciones $x_k = l \sum_{i=1}^k \sin \theta_i$ y $y_k = -l \sum_{i=1}^k \cos \theta_i$.

Es interesante comentar que, para conseguir que la simulación vaya a la misma velocidad que el tiempo real del sistema físico, es necesario introducir una herramienta de MATLAB llamada tic/toc, la cual permite calcular el tiempo real que tarda el procesamiento de gráficos y añadir una pausa variable en función de este tiempo para evitar aceleraciones.

9.1.3. Modos normales

Un modo normal de vibración es un patrón de movimiento en el que todas las partes de un sistema oscilan a la misma frecuencia y pasan por su posición de equilibrio exactamente al mismo tiempo. Cualquier movimiento que describa el sistema es una combinación lineal de estos modos. Dicho de otra forma, los modos normales forman un conjunto fundamental de soluciones del sistema.

Queremos encontrar soluciones donde todos los péndulos se muevan armónicamente a la misma frecuencia (ω), es decir, soluciones del tipo $\theta(t) = v \cos(\omega t)$. Si derivamos dos veces, la aceleración es $\ddot{\theta}(t) = -\omega^2 v \cos(\omega t)$. Teníamos que $\ddot{\theta} = -M^{-1}K\theta$. Si sustituimos esto en nuestra ecuación definiendo la matriz $A = M^{-1}K$:

$$\omega^2 v \cos(\omega t) = Av \cos(\omega t)$$

Podemos eliminar el término del coseno (ya que no siempre es cero), lo que nos deja con la ecuación:

$$Av = \omega^2 v$$

En álgebra lineal, esto es exactamente la definición de un problema de valores y vectores propios ($Ax = \lambda x$), donde $\lambda = \omega^2$ (VAPs) son las frecuencias al cuadrado y v (VEPs) son las formas de los modos.

Para n grandes, numéricamente, podemos encontrar estos valores y mostrar la animación de los modos normales. *Apéndice 4*

En un sistema no lineal, el concepto clásico de modo normal se rompe. En este caso, la frecuencia depende de la amplitud. Además, se destruye el principio de superposición, es decir, la combinación lineal de modos

no es una solución del sistema. Los modos ya no son independientes; se hablan entre sí, se transfieren energía y se distorsionan mutuamente.

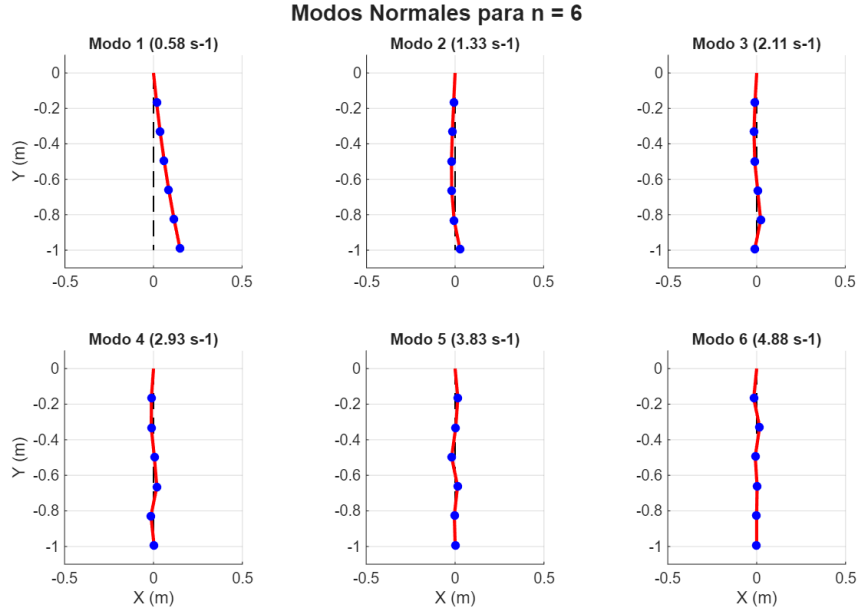


Figura 1: Modos normales para n=6

9.2. Simulación del sistema discreto no lineal

A pesar de haber expuesto únicamente el caso discreto lineal porque era el necesario para realizar el paso al medio continuo de la cuerda sin complicar enormemente las expresiones, también se puede obtener el sistema discreto no lineal válido para cualquier ángulo sin aproximaciones. No existen métodos analíticos para resolverlo, pero numéricamente es factible.

La energía potencial del sistema ya la habíamos obtenido en la Ecuación 3

$$U = -mgl \sum_{i=1}^n (n - i + 1) \cos \theta_i$$

Por otro lado, la ecuación de la energía cinética sin aproximaciones es la siguiente:

$$K = \frac{1}{2} ml^2 \sum_{i=1}^n \sum_{j=1}^n \min(i, j) \dot{\theta}_i \dot{\theta}_j \cos(\theta_i - \theta_j) \quad (52)$$

Así pues, la expresión de la Lagrangiana resulta en:

$$\mathcal{L} = K - U = \frac{1}{2} ml^2 \sum_{i=1}^n \sum_{j=1}^n \min(i, j) \dot{\theta}_i \dot{\theta}_j \cos(\theta_i - \theta_j) + mgl \sum_{i=1}^n (n - i + 1) \cos \theta_i \quad (53)$$

Ahora, aplicando la ecuación de Euler-Lagrange $\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}_k} \right) - \frac{\partial \mathcal{L}}{\partial \theta_k} = 0$ para cada ángulo (n grados de libertad) y reordenando los términos en forma matricial, resulta la ecuación siguiente:

$$M(\Theta) \ddot{\Theta} + C(\Theta, \dot{\Theta}) + G(\Theta) = 0 \quad (54)$$

donde $M(\Theta)$ es la matriz de masas, $C(\Theta, \dot{\Theta})$ es el vector que contiene las fuerzas centrífugas y de Coriolis y $G(\Theta)$ es el vector de fuerzas gravitacionales.

Las expresiones de sus elementos son las siguientes: $M_{ij}(\Theta) = ml^2 (n - \max(i, j) + 1) \cos(\theta_i - \theta_j)$
 $C_i(\Theta, \dot{\Theta}) = ml^2 \sum_{j=1}^n (n - \max(i, j) + 1) \sin(\theta_i - \theta_j) \dot{\theta}_j^2$ $G_i(\Theta) = mgl (n - i + 1) \sin \theta_i$

El siguiente paso es aislar las aceleraciones numéricamente $\ddot{\Theta} = M(\Theta)^{-1}(-C(\Theta, \dot{\Theta}) - G(\Theta))$ y, posteriormente, repetimos los mismos pasos que en el caso lineal para terminar la simulación. Una diferencia entre el caso lineal y el no lineal es que el segundo es más rígido, es decir, contiene variables que evolucionan lentamente y otras que oscilan rápidamente a la vez. En esta situación, el solucionador `ode15s` es más eficiente que el `ode45` porque usa métodos como la *Backward Differentiation Formula* que permiten establecer pasos más reducidos de tiempo.

9.3. Simulación del sistema continuo

Bloque 1: Parámetros Físicos y Configuración Inicial

Los parámetros físicos son la gravedad ($g = 9,81$) y la longitud de la cuerda.

- **f_inicial**: Define la forma de la cuerda en el instante $t = 0$ mediante una función explícita continua. Esta función debe cumplir que en el punto superior ($y=0$) sea cero (porque la cuerda está fijada) y no debe tener una pendiente pronunciada (sino la aproximación de ángulos pequeños no se cumple).
- **v_inicial**: Define la velocidad inicial de cada punto de la cuerda mediante una función de la posición vertical continua.

Bloque 2: Parámetros Numéricos

- **N_modos**: define la cantidad de armónicos (modos normales) de la función de Bessel para aproximar el movimiento. A mayor número, mayor precisión en los detalles rápidos.
- **N_cc**: Número de nodos para resolver las integrales matemáticas.

Bloque 3: Cálculo de Raíces de Bessel y Frecuencias

Para que la cuerda esté fija en el techo, la solución matemática exige encontrar dónde la función de Bessel $J_0(z)$ se hace cero (las raíces α_n).

El bucle **for** usa la función **fzero** para buscar numéricamente estas raíces (α_n) y luego calcula la frecuencia angular ω_n de cada modo usando la expresión ya encontrada:

$$\omega_n = \frac{\alpha_n}{2} \sqrt{\frac{g}{L}}$$

Bloque 4: Cuadratura de Clenshaw-Curtis

Llama a la función auxiliar del final para obtener los nodos (**y_cc**) y los pesos (**w_cc**).

Este es un método de integración numérica avanzado. En lugar de espaciar los puntos de forma equitativa (como la regla del trapecio), los concentra en los extremos del intervalo usando una distribución basada en cosenos (nodos de Chebyshev). Esto evita errores numéricos graves conocidos como el fenómeno de Runge.

Bloque 5: Cálculo de los Coeficientes A_n y B_n (Análisis de Fourier-Bessel)

Cualquier estado inicial se puede escribir como una combinación lineal de modos:

$$f(y) = \sum A_n J_0 \left(\alpha_n \sqrt{\frac{L-y}{L}} \right)$$

Para hallar A_n (amplitud de posición) y B_n (amplitud de velocidad), el código calcula una integral (un producto escalar) para cada modo n .

- **J0_kernel**: Es la forma del modo n a lo largo de la cuerda.

- **denominador:** Es la constante de normalización debida a la ortogonalidad de las funciones de Bessel.
- **sum(w_cc .* f_nodos .* J0_kernel):** Realiza la integral mediante una suma pesada usando los pesos de Clenshaw–Curtis.

Bloque 6: Evaluación y Bucle de Animación

Para optimizar el código, precalculamos una matriz que contiene la forma espacial de todos los modos.

Dentro del bucle temporal se realizan los siguientes pasos:

- **Evolución temporal:** Multiplica cada coeficiente por su respectivo $\cos(\omega_n t)$ y $\sin(\omega_n t)$.
- **Reconstrucción:** Multiplica la matriz de modos por el vector temporal. Esto realiza la suma de todos los armónicos instantáneamente mediante multiplicación matricial.
- **Gráfica:** Dibuja la cuerda en la posición actual. El signo negativo en $-y_{vis}$ se usa para que el gráfico apunte hacia abajo (ya que la cuerda cuelga del techo, $y = 0$). Creamos el bucle de animación con el comando `drawnow` que actualiza la gráfica en cada instante.

9.4. Error numérico. Sensibilidad a CI.

Toda simulación numérica conlleva un error numérico debido a la precisión finita (aunque elevada) de cualquier ordenador. A este error, se le añade la tolerancia asignada a los métodos numéricos de integración numérica (Clenshaw-Curtis) y de resolución de sistemas de EDOs (Runge-Kutta). Sin embargo, aún se pueden conseguir de forma razonable precisiones de $1e-8$.

El caso discreto para ángulos pequeños, al ser lineal, es fácilmente predecible y numéricamente estable. Pequeñas variaciones de las condiciones iniciales dan lugar a movimientos de comportamiento similar. No obstante, no sucede lo mismo con el caso discreto para cualquier ángulo. Cuando el ángulo aumenta, es decir, cuando la energía aumenta, el sistema a resolver es altamente no lineal, por lo que se vuelve extremadamente sensible a las condiciones iniciales. Esto significa que variaciones infinitesimales en estas dan lugar a trayectorias completamente distintas. En este caso, se dice que el sistema es caótico porque no se puede predecir su evolución con precisión. Efectivamente, podemos observar el error numérico existente configurando la posición inicial de los péndulos en los puntos de equilibrio inestable (verticalmente hacia arriba), ya que en esta situación, el conjunto debería quedarse quieto, pero el error numérico hace que caiga.

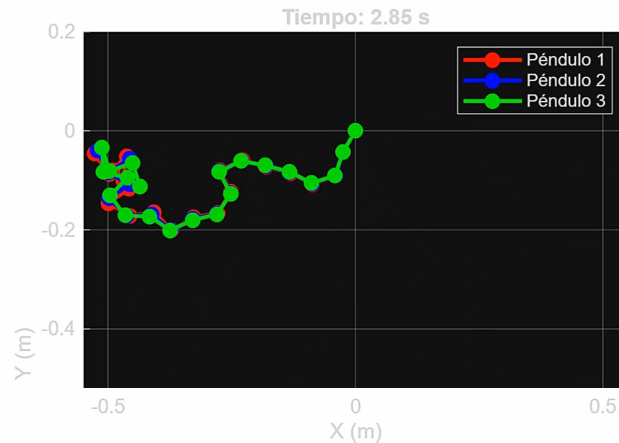


Figura 2: Posición al cabo de tres segundos de tres péndulos múltiples idénticos con posiciones iniciales modificadas con una diferencia de $1e-8$. Se observa que esta pequeña perturbación conduce a trayectorias radicalmente distintas en sólo tres segundos.

En cuanto al caso continuo, al proceder del sistema lineal y al estar gobernado por una EDP lineal

resultante de la aproximación de ángulos pequeños, también es poco sensible a las condiciones iniciales. Aún así, se produce un mayor error numérico porque es necesario encontrar las raíces de la función de Bessel de primera especie y orden cero y proyectar mediante integración numérica la función posición inicial. Además, se puede regular la precisión aumentando la cantidad de modos normales de la función de Bessel, aunque se debe buscar el equilibrio: las oscilaciones de cada modo son más intensas, y si se aumentan los modos se tiene que aumentar también la cantidad de nodos de Chebyshev para capturar los detalles de los modos y evitar errores mayores (*aliasing* numérico).

9.4.1. Péndulo doble. Espacio de fases y sección de Poincaré.

Podemos profundizar en el estudio de la teoría del caos de un péndulo múltiple (sistema no lineal) de forma intuitiva limitándonos al caso de dos péndulos, el cual nos va a permitir observar el espacio de fases 3D y analizar el comportamiento impredecible de sus trayectorias en función de la energía.

Aunque hemos comentado que el sistema no lineal es altamente caótico, se pueden observar grados de orden dentro del caos. Cuando fijamos un valor concreto de la energía total del sistema, hay trayectorias más propensas al caos y otras que siguen movimientos de frecuencia predecible en función de las condiciones iniciales (coexistencia en el espacio de fases). Si se distribuye la energía de manera que los dos péndulos se muevan en fase, el sistema se comportará de forma regular y periódica. En cambio, si se deja caer el primer péndulo desde 90° mientras el segundo está quieto, la energía empieza a transferirse de forma violenta y asimétrica entre ambos. Además, si la energía total del sistema aumenta, también aumenta la cantidad de trayectorias caóticas.

Estos fenómenos quedan recogidos en el Teorema de KAM (Kolmogorov-Arnold-Moser), que establece que si a un sistema dinámico perfectamente ordenado y predecible (sistema integrable, cuyas órbitas forman trayectorias cerradas en forma de "donut" llamadas toros) se le aplica una perturbación pequeña, el sistema no cae inmediatamente en el caos absoluto, sino que la mayoría de estas órbitas estables sobreviven simplemente deformándose. La clave de esta estabilidad depende de la relación entre las frecuencias del sistema: aquellas órbitas con frecuencias fuertemente irracionales (que no entran en sincronía periódica) repelen las perturbaciones y preservan su estructura ordenada para siempre, mientras que aquellas con frecuencias racionales o resonantes se destruyen, abriendo paso a regiones localizadas de comportamiento caótico.

Para mostrar lo expuesto, moldearemos el espacio de fases del sistema numéricamente y usaremos la herramienta de la sección de Poincaré. El espacio de fases es un espacio geométrico donde cada punto representa un estado completo del sistema. En el caso del péndulo doble, forma un hiperespacio de 4D $(\theta_1, \theta_2, \omega_1, \omega_2)$, aunque como la energía total se conserva (es un sistema Hamiltoniano), las trayectorias quedan atrapadas en las superficies de nivel 3D de cada valor energético. Trazando una órbita en este espacio, se puede observar las estructuras llamadas "toros de KAM", a la vez, cómo desaparecen cuando la energía aumenta.

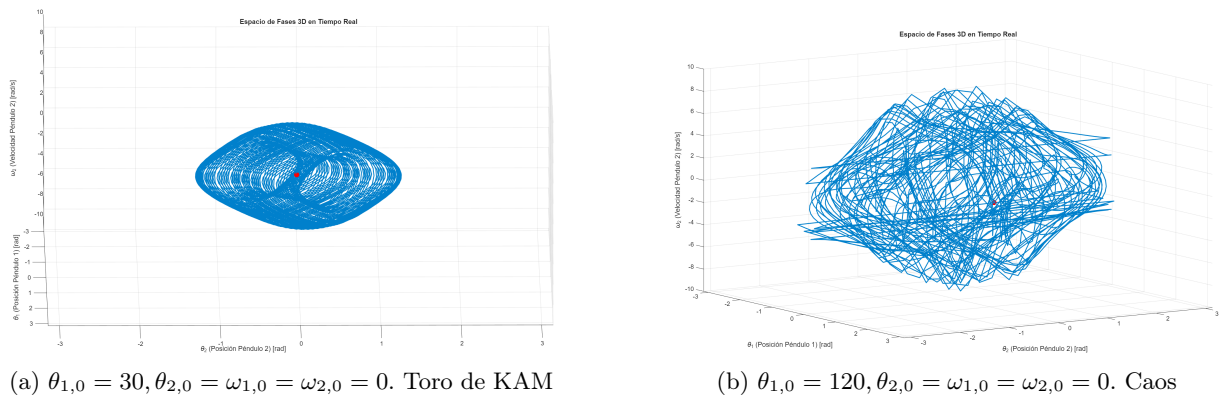


Figura 3: Comparación entre espacios de fase con energías distintas

Para analizar mejor el caos de estas órbitas (y poder hacerlo en dimensiones superiores) se usa la sección de Poincaré, que consiste en tomar un plano paralelo a una de las variables del espacio de fases, por ejemplo el plano $\theta_1 = 0$ y graficar los puntos en los cuales la trayectoria interseca este plano. Además, imponemos que solo se registren los puntos con $\omega_1 > 0$ para obtener más claridad. Físicamente, significa dibujar un punto en la pantalla cuando el primer péndulo pase exactamente por la vertical apuntando hacia abajo, y además se esté moviendo hacia la derecha. Esto nos permite tener una visualización en 2D y observar las islas de estabilidad.

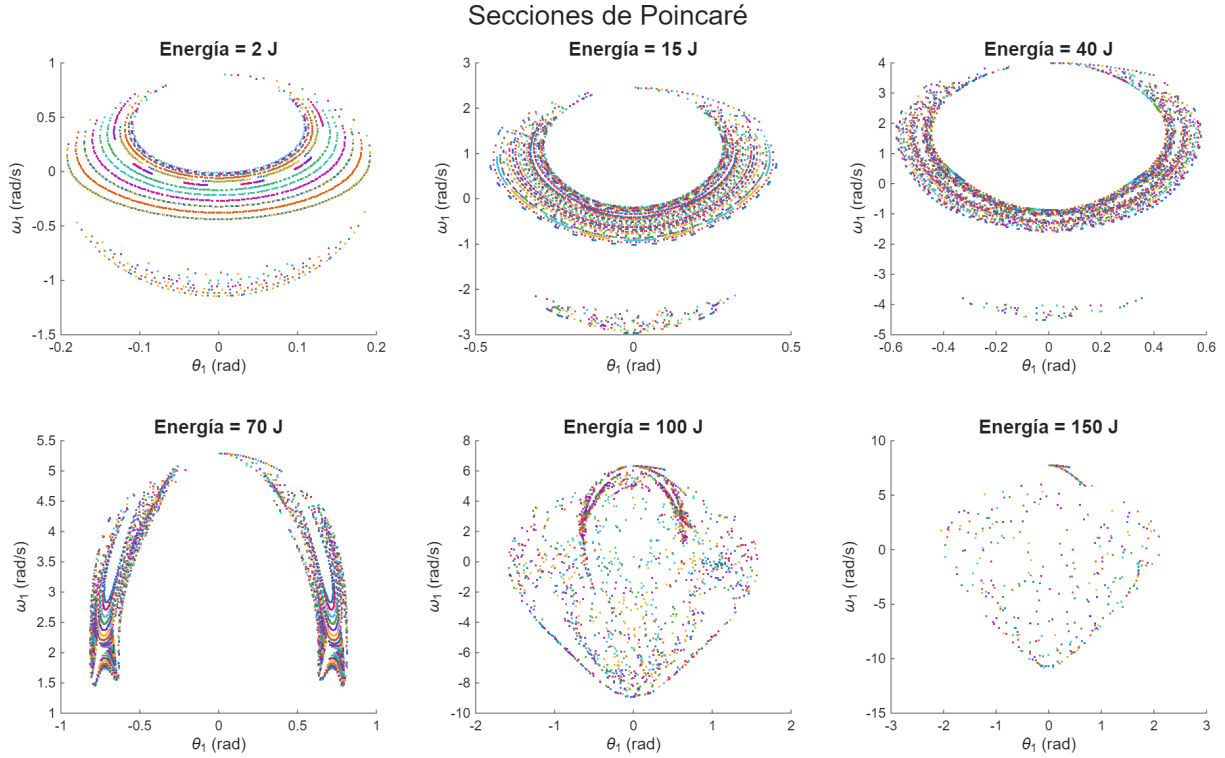


Figura 4: Secciones de Poincaré para distintos niveles energéticos. Cada gráfico está compuesto por las secciones la Poincaré de varias trayectorias diferentes (cada una ilustrada con un color distinto) pertenecientes al mismo nivel de energía.

Observamos que para una misma energía del sistema hay trayectorias predecibles y otras caóticas como hemos descrito anteriormente, y que cuanto más energía el sistema se vuelve completamente caótico. Este límite entre orden y caos queda determinado por la separatriz entre las órbitas que oscilan y las que pueden rotar.

10. Análisis del sistema discreto no lineal

Antes de analizar los puntos de equilibrio, se plantean las relaciones geométricas y la energía potencial total del sistema de péndulos:

$$y = l \cos \theta, \quad \dot{y} = -l \sin \theta \dot{\theta} \quad (55)$$

$$x = l \sin \theta, \quad \dot{x} = l \cos \theta \dot{\theta} \quad (56)$$

La energía potencial de la masa k -ésima es:

$$U_k = -mgy_k = -mgl \sum_{i=1}^k \cos \theta_i \quad (57)$$

Sumando para todo el sistema se obtiene la energía potencial total U :

$$U = \sum_{k=1}^n U_k = -mgl \sum_{k=1}^n \sum_{i=1}^k \cos \theta_i \quad (58)$$

$$U = -mgl \sum_{i=1}^n (n - i + 1) \cos \theta_i \quad (59)$$

10.1. Puntos de Equilibrio (PEQs)

Está en equilibrio si $\frac{\partial U}{\partial \theta_j} = 0$ y $\dot{v} = 0$, por lo que si $F_g = \nabla U$, en estas condiciones no tiene ninguna velocidad ni aceleración, por lo que se queda quieto.

Calculando la derivada de la energía potencial:

$$\frac{\partial U}{\partial \theta_j} = mgl(n - j + 1) \sin \theta_j \implies \sin \theta_j = 0 \implies \begin{cases} \theta = 0 \\ \theta = \pi \end{cases} \quad (60)$$

Cada péndulo puede elegir de entre 2 estados, por lo que hay un total de:

$$\underbrace{2 \cdot 2 \cdot 2 \dots}_{n \text{ veces}} = 2^n \text{ PEQs} \quad (61)$$

El equilibrio trivial (todos los ángulos en cero) se expresa como:

$$\Theta_0^* = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad (62)$$

10.2. Estabilidad

Para estudiar la estabilidad evaluamos los elementos de la matriz Hessiana H_{jk} :

$$H_{jk} = \frac{\partial^2 V}{\partial \theta_j \partial \theta_k} \implies \frac{\partial [mgl(n - j + 1) \sin \theta_j]}{\partial \theta_k} \implies \begin{cases} = 0 & k \neq j \\ = mgl(n - j + 1) \cos \theta_j & k = j \end{cases} \quad (63)$$

La matriz Hessiana general H resulta:

$$H = \begin{pmatrix} mgl n \cos \theta_1 & 0 & 0 & \dots & 0 \\ 0 & mgl(n-1) \cos \theta_2 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & mgl \cos \theta_n \end{pmatrix} \quad (64)$$

La matriz es diagonal, por lo que los vap (valores propios) son los propios elementos de la diagonal:

- **Mínimo** ($\theta_1 = \theta_2 = \dots = \theta_n = 0$): $\implies$ PEQ estable (todos los vap ≥ 0) $\rightarrow$ atractor.
- **Máximo** ($\theta_1 = \theta_2 = \dots = \theta_n = \pi$): $\implies$ PEQ inestable (todos los vap ≤ 0) $\rightarrow$ repulsor.
- **Punto de silla** ($\theta_j = \dots = \theta_k = 0$, $\theta_i = \dots = \theta_l = \pi$ con al menos uno en cada grupo): $\implies$ PEQ inestable no repulsor (hay $2^n - 2$).

Fuentes

Funciones de Bessel – UPV/EHU

Universidad del País Vasco. (s.f.). Funciones de Bessel.

<http://www.sc.ehu.es/sbweb/fisica3/especial/bessel/bessel.html>

Ecuaciones diferenciales en MATLAB – MathWorks

MathWorks. (s.f.). Differential equations.

<https://es.mathworks.com/help/matlab/math/differential-equations.html>

Métodos de Runge–Kutta – AcademiaLab

AcademiaLab. (s.f.). Métodos de Runge–Kutta.

<https://academia-lab.com/enciclopedia/metodos-de-runge-kutta/>

A. Código 1: Simulación discreta para ángulos pequeños (sistema lineal)

```

1 %% Caso discreto para ángulos pequeños
2 close all;
3 % PARÁMETROS FÍSICOS
4 n = 20; % Número de péndulos
5 L = 1; % Longitud total (m)
6 M_total = 20; % Masa total (kg)
7 g = 9.81; % Gravedad (m/s^2)
8
9 l = L/n; % Longitud de cada segmento
10 m = M_total/n; % Masa de cada partícula
11
12 % 1. CONSTRUCCIÓN DE MATRICES (M y K)
13 K = zeros(n,n);
14 M = zeros(n,n);
15 for i = 1:n
16     K(i,i) = m * g * l * (n - i + 1);
17     for j = 1:n
18         M(i,j) = m * l^2 * (n - max(i,j) + 1);
19     end
20 end
21
22 % 2. CONFIGURACIÓN DE LA SIMULACIÓN
23 tspan = 0:0.05:20; % Tiempo de 0 a 30s con pasos de 0.05s
24 theta0 = [0.17*ones(n,1)]; % Posiciones iniciales
25 omega0 = [zeros(n,1)]; % Velocidades iniciales
26
27 y0 = [theta0; omega0]; % Estado inicial [ángulos; velocidades]
28
29 % Definición del sistema de EDOs: y' = [velocidades; aceleraciones]
30 sistema = @(t, y) [y(n+1:end); -M \ (K * y(1:n))];
31
32 % Resolver las ecuaciones
33 [t, y_out] = ode45(sistema, tspan, y0);
34 thetas = y_out(:, 1:n);
35
36
37 %% VIDEO
38 opcion_video = questdlg('¿Deseas guardar la animación en un archivo de video MP4?', ...
39     'Guardar Video', ...
40     'Sí', 'No', 'No'); % 'No' es la opción por defecto
41
42 guardar_video = strcmp(opcion_video, 'Sí');
43
44 if guardar_video
45     nombre_archivo = sprintf('Sistema_Lineal_n%d.mp4', n);
46     v = VideoWriter(nombre_archivo, 'MPEG-4');
47     v.FrameRate = (size(tspan,2)-1)/tspan(end); % fotogramas por segundo
48     v.Quality = 95; % Alta calidad de compresión
49     open(v);
50     disp(['Grabando video... El archivo se guardará como: ', nombre_archivo]);
51 end
52 %%
53
54 % 3. ANIMACIÓN
55 figure('Color', 'w');
56 grid on;
57 hold on;
58 axis equal;
59 % Ajustar los límites del gráfico
60 xlim([-L*0.6, L*0.6]);
61 ylim([-L*1.1, L*0.1]);
62 xlabel('X (m)'); ylabel('Y (m)');
63
64
65 % Crear el objeto gráfico de la cuerda (línea con círculos)
66 hCuerda = plot(0, 0, '-or', 'LineWidth', 2, 'MarkerFaceColor', 'r');
67

```

```

68 % Bucle de frames para el movimiento
69 for k = 1:length(t)
70     tk=tspan(k);
71     title(sprintf('Simulación del sistema lineal (n = %5.0f)\nt = %5.2f s',n,tk));
72     tic;
73     % Calcular posiciones cartesianas (x, y) a partir de los ángulos
74     % x = l * sin(theta1) + l * sin(theta2) ...
75     current_thetas = thetas(k, :);
76     posX = [0, cumsum(l * sin(current_thetas))];
77     posY = [0, cumsum(-l * cos(current_thetas))];
78
79     % Actualizar datos del gráfico
80     set(hCuerda, 'XData', posX, 'YData', posY);
81
82     % Forzar dibujado y pequeña pausa (que depende del tiempo de procesado) para
      velocidad real
83     drawnow;
84
85     % Captura el frame actual de la figura
86     if guardar_video
87         frame = getframe(gcf);
88         writeVideo(v, frame);
89     end
90     tproces=toc;
91     pause(max(0,0.05-tproces));
92 end
93
94 if guardar_video
95     close(v);
96     disp('Video guardado y exportado con éxito!');
97 end

```

B. Código 2: Simulación discreta (sistema no lineal)

```

1  %% Caso_discreto_no_lineal
2  close all;
3  % --- Configuración del Sistema ---
4  n = 20; % Número de péndulos
5  L_total = 1.0; % Longitud total (metros)
6  M_total = 20.0; % Masa total (kg)
7  g = 9.81;
8
9  l = L_total / n;
10 m = M_total / n;
11
12 % --- Condiciones Iniciales ---
13
14 % Ángulos iniciales: theta_i_0 en radianes
15 % Ejemplos: una curva suave inicial (senoidal) 0.5 * sin(linspace(0, pi/2, n));
16 % theta0=zeros(n,1); theta0(1)= deg2rad(45); % Solo el primero a 45 grados
17
18 theta0 = deg2rad(90)*ones(n, 1); % Todos en 90
19 omega0 = zeros(n, 1); % Todas en 0 s-1
20 z0 = [theta0; omega0];
21
22 % --- Simulación ---
23 tspan = 0:0.05:20; % Pasos de tiempo fijos para la animación
24 opts = odeset('RelTol', 1e-6, 'AbsTol', 1e-8);
25 % El sistema está programado en una función aparte.
26 [t, z] = ode45(@(t, z) sistema_derivadas(t, z, n, l, m, g), tspan, z0, opts);
27
28 %--- Vídeo ---
29 opcion_video = questdlg('¿Deseas guardar la animación en un archivo de video MP4?',
30     ...
31     'Guardar Video', ...
32     'Si','No','No'); % 'No' es la opción por defecto

```

```

33 guardar_video = strcmp(opcion_video, 'Sí');
34
35 if guardar_video
36 nombre_archivo = sprintf('Sistema_No_Lineal_n%d.mp4', n);
37 v = VideoWriter(nombre_archivo, 'MPEG-4');
38 v.FrameRate = (size(tspan,2)-1)/tspan(end); % fotogramas por segundo
39 v.Quality = 95; % Alta calidad de compresión
40 open(v);
41 disp(['Grabando video... El archivo se guardará como: ', nombre_archivo]);
42 end
43
44 % --- Animación ---
45 figure('Color', 'w', 'Name', sprintf('Simulación n=%5.0f',n));
46 h_plot = plot(0, 0, '-o', 'LineWidth', 2, 'MarkerFaceColor', 'r');
47 grid on;
48 axis equal;
49 % Ajustar ejes según longitud total
50 limit = L_total * 1.1;
51 axis([-limit/2, limit/2, -limit, 0.2]);
52 xlabel('X (m)'); ylabel('Y (m)');
53
54 for k = 1:length(t)
55 tic;
56 % Extraer ángulos en el instante k
57 thetas = z(k, 1:n);
58
59 % Calcular posiciones cartesianas de cada masa (sumatoria acumulada)
60 x = [0; cumsum(l * sin(thetas'))];
61 y = [0; cumsum(-l * cos(thetas'))];
62
63 % Actualizar gráfico
64 set(h_plot, 'XData', x, 'YData', y);
65 title(sprintf('Simulación del sistema no lineal\nTiempo: %.2f s', t(k)));
66 drawnow;
67
68 if guardar_video
69 frame = getframe(gcf);
70 writeVideo(v, frame);
71 end
72
73 tproces=toc;
74 pause(max(0,0.05-tproces));
75 end
76
77 if guardar_video
78 close(v);
79 disp('Video guardado y exportado con éxito!');
80 end
81
82 %% Definimos el sistema en una función aparte
83 function dzdt = sistema_derivadas(~, z, n, l, m, g)
84 theta = z(1:n);
85 omega = z(n+1:end);
86
87 M_mat = zeros(n, n);
88 F = zeros(n, 1);
89
90 % Llenado de matrices basado en Euler-Lagrange
91 for m_idx = 1:n
92 % Fuerza restauradora de la gravedad
93 F(m_idx) = -m * g * l * (n - m_idx + 1) * sin(theta(m_idx));
94
95 for j = 1:n
96 % Coeficiente de inercia acoplada
97 % Representa la masa total que cuelga por debajo de los nodos m y j
98 A_mj = (n - max(m_idx, j) + 1) * m * l^2;
99
100 % Matriz de Masa: M(theta) * ddot{theta}
101 M_mat(m_idx, j) = A_mj * cos(theta(m_idx) - theta(j));
102
103 % Términos centrífugos: A_mj * dot{theta}^2 * sin(theta_m - theta_j)

```

```

104         if m_idx ~ j
105             F(m_idx) = F(m_idx) - A_mj * omega(j)^2 * sin(theta(m_idx) - theta(j));
106         end
107     end
108 end
109
110 % Resolver sistema lineal para aceleraciones
111 alpha = M_mat \ F;
112 dzdt = [omega; alpha];
113 end

```

C. Código 3: Simulación continua (funciones de Bessel)

```

1  %% Caso continuo (ángulos pequeños)
2  % =====
3  % PARAMETROS FÍSICOS
4  % =====
5  clear vars;
6  clc;
7  close all;
8  g_grav = 9.81; % Aceleración de la gravedad (m/s^2)
9  L = 1.0; % Longitud de la cuerda (m)
10
11 % =====
12 % CONFIGURACIÓN DE LAS CONDICIONES INICIALES
13 % =====
14 % f(y): Posición inicial. Ejemplo: Cuerda desplazada en el centro (tipo pulso)
15 f_inicial = @(y) 0.01 * exp(-15 * (y - L/2).^2);
16
17 % g(y): Velocidad inicial. Ejemplo: Perfil de velocidad tras un impacto: -0.2 * (y /
18 L);
19 v_inicial = @(y) zeros(size(y));
20
21 % =====
22 % PARÁMETROS NUMÉRICOS
23 % =====
24 N_modos = 25; % Mayor número de modos para capturar transitorios complejos
25 N_cc = 64; % Número de intervalos de Chebyshev (debe ser par)
26 N_visual = 200; % Puntos para la renderización de la cuerda
27 t_max = 20; % Tiempo total de simulación (s)
28 dt = 0.05; % Paso de tiempo de la animación (s)
29
30 y_vis = linspace(0, L, N_visual);
31 t = 0:dt:t_max;
32
33 % =====
34 % 1. CÁLCULO DE RAÍCES DE BESSEL (alpha_n) Y FRECUENCIAS (omega_n)
35 % =====
36 alpha = zeros(N_modos, 1);
37 omega = zeros(N_modos, 1);
38 for n = 1:N_modos
39     estimacion = pi * (n - 0.25);
40     alpha(n) = fzero(@(z) besselj(0, z), estimacion);
41     omega(n) = (alpha(n) / 2) * sqrt(g_grav / L);
42 end
43
44 % =====
45 % 2. OBTENCIÓN DE NODOS Y PESOS DE CLENSHAW-CURTIS
46 % =====
47 [y_cc, w_cc] = pesos_cleenshaw_curtis(N_cc, L);
48
49 % =====
50 % 3. CÁLCULO DE COEFICIENTES A_n Y B_n USANDO CUADRATURA
51 % =====
52 A = zeros(N_modos, 1);
53 B = zeros(N_modos, 1);

```

```

54 % Evaluamos las funciones iniciales en los nodos de Chebyshev
55 f_nodos = f_inicial(y_cc);
56 v_nodos = v_inicial(y_cc);
57
58 % Asegurar que sean vectores columna para evitar conflictos de dimensiones
59 f_nodos = f_nodos(:);
60 v_nodos = v_nodos(:);
61 w_cc = w_cc(:);
62 y_cc = y_cc(:);
63
64 for n = 1:N_modos
65     % Kernel espacial de Bessel para el modo n
66     J0_kernel = besselj(0, alpha(n) * sqrt((L - y_cc) / L));
67
68     % Denominador común por ortogonalidad
69     denominador = L * (besselj(1, alpha(n)))^2;
70
71     % Integración por Clenshaw-Curtis (Suma pesada / producto escalar)
72     integral_A = sum(w_cc .* f_nodos .* J0_kernel);
73     integral_B = sum(w_cc .* v_nodos .* J0_kernel);
74
75     % Asignación de coeficientes espectrales
76     A(n) = integral_A / denominador;
77     B(n) = integral_B / (omega(n) * denominador);
78 end
79
80 % =====
81 % VÍDEO
82 % =====
83 opcion_video = questdlg('¿Deseas guardar la animación en un archivo de video MP4?',
84     ...
85     'Guardar Video', ...
86     'Sí', 'No', 'No'); % 'No' es la opción por defecto
87
88 guardar_video = strcmp(opcion_video, 'Sí');
89
90 if guardar_video
91     nombre_archivo = sprintf('Sistema_Continuo.mp4');
92     v = VideoWriter(nombre_archivo, 'MPEG-4');
93     v.FrameRate = 1/dt; % fotogramas por segundo
94     v.Quality = 95; % Alta calidad de compresión
95     open(v);
96     disp(['Grabando video... El archivo se guardará como: ', nombre_archivo]);
97 end
98
99 % =====
100 % 4. EVALUACIÓN DE LA SOLUCIÓN Y BUCLE DE ANIMACIÓN
101 % =====
102 figure('Color', [1 1 1], 'Position', [200, 100, 550, 650]);
103
104 % Precalculamos la matriz espacial de modos para acelerar el bucle
105 Modos_Matriz = zeros(N_visual, N_modos);
106 for n = 1:N_modos
107     Modos_Matriz(:, n) = besselj(0, alpha(n) * sqrt((L - y_vis) / L));
108 end
109
110 for k = 1:length(t)
111     tk = t(k);
112     tic;
113     % Combinación lineal de las soluciones temporales
114     temp_A = A .* cos(omega * tk);
115     temp_B = B .* sin(omega * tk);
116     componente_temporal = temp_A + temp_B;
117
118     % Multiplicación matricial para obtener x(y) en el instante t_k
119     x_yt = Modos_Matriz * componente_temporal;
120
121     % Gráfica
122     plot(x_yt, -y_vis, 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2.5);
123     hold on;

```

```

123     plot(0, 0, 'ks', 'MarkerFaceColor', [0.6350, 0.0780, 0.1840], 'MarkerSize', 8); %
124         Soporte
125     hold off;
126
127     % Estética del gráfico
128     grid on;
129     xlim([-0.25, 0.25]);
130     ylim([-L - 0.1, 0.1]);
131     xlabel('Desplazamiento Horizontal x [m]');
132     ylabel('Longitud de la Cuerda y [m] (0 = Techo)');
133     title(sprintf('Cuerda Continua\nt = %.2f s', tk));
134
135     drawnow limitrate;
136
137     if guardar_video
138         frame = getframe(gcf);
139         writeVideo(v, frame);
140     end
141
142     t_p=toc;
143     pause(dt-t_p);
144 end
145
146 if guardar_video
147     close(v);
148     disp('¡Video guardado y exportado con éxito!');
149 end
150
151 % =====
152 % FUNCIÓN AUXILIAR: Cuadratura de Clenshaw-Curtis
153 % =====
154 function [y_cc, w_cc] = pesos_clenshaw_curtis(N, L)
155     % Forzar a que N sea par
156     if mod(N, 2) ~= 0, N = N + 1; end
157
158     % 1. Nodos de Chebyshev en [-1, 1]
159     theta = pi * (0:N)' / N;
160     x_cheb = cos(theta);
161
162     % Mapeo de nodos al intervalo físico de la cuerda [0, L]
163     y_cc = (L / 2) * (x_cheb + 1);
164
165     % 2. Cálculo directo de los pesos estándar en [-1, 1]
166     w_std = zeros(N + 1, 1);
167     for i = 0:N
168         sum_terms = 0;
169         for j = 1:(N/2 - 1)
170             sum_terms = sum_terms + (2 / (1 - 4*j^2)) * cos(2*j*i*pi / N);
171         end
172         % Añadir los términos extremos de la sumatoria analítica
173         val_izq = 1;
174         val_der = cos(i*pi) / (1 - N^2);
175
176         w_std(i+1) = (2 / N) * (0.5 * val_izq + sum_terms + 0.5 * val_der);
177     end
178
179     % Ajustar extremos del intervalo
180     w_std(1) = w_std(1) / 2;
181     w_std(end) = w_std(end) / 2;
182
183     % Escalar pesos al tamaño del dominio [0, L]
184     w_cc = w_std * (L / 2);
185
186     % Invertir vectores para que comiencen físicamente en y = 0 (anclaje fijo)
187     y_cc = flipud(y_cc);
188     w_cc = flipud(w_cc);
189 end

```

D. Código 4: Animación de los modos normales (sistema lineal)

```

1 %% Animación Simultánea de Modos Normales (del sistema lineal)
2 clear; clc; close all;
3
4 % PARÁMETROS FÍSICOS
5 n = 6; % # péndulos
6 L = 1; % Longitud total (m)
7 M_total = 20; % Masa total (kg)
8 g = 9.81; % Gravedad (m/s^2)
9 l = L/n; % Longitud de cada segmento
10 m = M_total/n; % Masa de cada partícula
11
12 % 1. CONSTRUCCIÓN DE MATRICES (M y K)
13 K = zeros(n,n);
14 M = zeros(n,n);
15 for i = 1:n
16     K(i,i) = m * g * l * (n - i + 1);
17     for j = 1:n
18         M(i,j) = m * l^2 * (n - max(i,j) + 1);
19     end
20 end
21
22 % 2. CÁLCULO DE MODOS PROPIOS (VAPs y VEPs)
23 [V, D] = eig(K, M);
24 frecuencias_angulares = sqrt(diag(D));
25
26 % Ordenar modos de menor a mayor frecuencia
27 [frecuencias_angulares, idx] = sort(frecuencias_angulares);
28 V = V(:, idx);
29
30 % =====
31 % PREGUNTA INTERACTIVA PARA GUARDAR VIDEO
32 % =====
33 opcion_video = questdlg('¿Deseas guardar la animación en un archivo de video MP4?', ...
34     'Guardar Video', ...
35     'Sí', 'No', 'No'); % 'No' es la opción por defecto
36
37 guardar_video = strcmp(opcion_video, 'Sí');
38
39 if guardar_video
40     nombre_archivo = sprintf('Modos_Normales_n%d.mp4', n);
41     v = VideoWriter(nombre_archivo, 'MPEG-4');
42     v.FrameRate = 50; % 50 fotogramas por segundo
43     v.Quality = 95; % Alta calidad de compresión
44     open(v);
45     disp(['Grabando video... El archivo se guardará como: ', nombre_archivo]);
46 end
47 % =====
48
49 % 3. CONFIGURACIÓN DE LA FIGURA Y SUBPLOTS
50 fig = figure('Color', 'w', 'Position', [100, 100, 900, 600]);
51
52 modos_a_graficar = min(n, 12);
53 columnas = ceil(sqrt(modos_a_graficar));
54 filas = ceil(modos_a_graficar / columnas);
55
56 amplitud_visual = 0.25;
57 hCuerdas = cell(modos_a_graficar, 1);
58
59 % Dibujado inicial de los subplots
60 for modo = 1:modos_a_graficar
61     subplot(filas, columnas, modo);
62     grid on; hold on; axis equal;
63
64     % Línea de equilibrio de fondo
65     plot([0, 0], [0, -L], '--', 'Color', [0.5 0.5 0.5], 'LineWidth', 1);
66
67     % Crear el objeto gráfico animable
68     hCuerdas{modo} = plot(zeros(n+1,1), -(0:1:L), '-or', ...
69         'LineWidth', 2, 'MarkerFaceColor', 'b', 'MarkerSize', 4);

```

```

70     xlim([-L*0.6, L*0.6]);
71     ylim([-L*1.1, L*0.1]);
72
73     f_hz = frecuencias_angulares(modos) / (2*pi);
74     title(sprintf('Modo %d (%.2f s-1)', modos, f_hz), 'FontSize', 10);
75
76     if modos > (filas-1)*columnas, xlabel('X (m)'); end
77     if mod(modos, columnas) == 1, ylabel('Y (m)'); end
78 end
79
80 sgtitle(sprintf('Movimiento de los Modos Normales (n = %d)', n), ...
81     'FontSize', 14, 'FontWeight', 'bold');
82
83 % 4. BUCLE DE ANIMACIÓN
84 t_anim = 0:0.02:10;
85 fps_control = tic;
86
87 for k = 1:length(t_anim)
88     t = t_anim(k);
89
90     % Actualizar cada modo individualmente
91     for modo = 1:modos_a_graficar
92         theta_actual = amplitud_visual * (V(:, modo) / max(abs(V(:, modo)))) * cos(
93             frecuencias_angulares(modos) * t);
94
95         posX = [0; cumsum(1 * sin(theta_actual))];
96         posY = [0; cumsum(-1 * cos(theta_actual))];
97
98         set(hCuerdas{modo}, 'XData', posX, 'YData', posY);
99     end
100
101     % Forzar el renderizado en pantalla
102     drawnow;
103
104     % Captura el frame actual de la figura
105     if guardar_video
106         frame = getframe(fig);
107         writeVideo(v, frame);
108     end
109
110     % Sincronización para la reproducción en tiempo real
111     t_procesado = toc(fps_control);
112     pause(max(0, 0.02 - t_procesado));
113     fps_control = tic;
114 end
115
116 % Cierre del archivo de video
117 if guardar_video
118     close(v);
119     disp('Video guardado y exportado con éxito!');
120 end

```

E. Código 5: Espacio de fases 3D del sistema no lineal

```

1 function Codigo5_espacio_de_fases()
2 % Representación del espacio de fases 3D del doble péndulo considerando la
3 % energía constante
4 close all;
5 % --- PARÁMETROS FÍSICOS ---
6 g = 9.81;
7 l = 1.0;
8 m = 20.0;
9
10 % --- CONDICIONES INICIALES ---
11 theta1_0 = deg2rad(80);
12 theta2_0 = deg2rad(0);

```



```

13 omega1_0 = 0;
14 omega2_0 = 0;
15 z = [theta1_0; theta2_0; omega1_0; omega2_0]; % Vector de estado inicial
16
17 % --- CONFIGURACIÓN DE LA ANIMACIÓN ---
18 t_total = 150; % Tiempo total de simulación
19 dt = 0.05; % Intervalo de tiempo para cada paso de animación
20
21 % ---Video
22 opcion_video = questdlg('¿Deseas guardar la animación en un archivo de video MP4?',
23 ...
24 'Guardar Video', ...
25 'Sí','No','No'); % 'No' es la opción por defecto
26
27 guardar_video = strcmp(opcion_video, 'Sí');
28
29 if guardar_video
30 nombre_archivo = sprintf('Espacio_Fases.mp4');
31 v = VideoWriter(nombre_archivo, 'MPEG-4');
32 v.FrameRate = 1/dt; % fotogramas por segundo
33 v.Quality = 95; % Alta calidad de compresión
34 open(v);
35 disp(['Grabando video... El archivo se guardará como: ', nombre_archivo]);
36 end
37 % ---
38
39 % Configuración de la ventana gráfica
40 figure('Position', [200, 100, 800, 600], 'Color', 'w');
41
42 % Creamos una línea animada en 3D (el camino del espacio de fases)
43 h_trayectoria = animatedline('Color', [0 0.5 0.8], 'LineWidth', 1.5);
44
45 % Creamos un punto que representa el "estado actual" del péndulo
46 hold on;
47 h_estado_actual = plot3(z(1), z(2), z(4), 'ro', 'MarkerFaceColor', 'r', 'MarkerSize',
48 8);
49 hold off;
50
51 % Configuración de los ejes y etiquetas
52 xlabel('\theta_1 (Posición Péndulo 1) [rad]');
53 ylabel('\theta_2 (Posición Péndulo 2) [rad]');
54 zlabel('\omega_2 (Velocidad Péndulo 2) [rad/s]');
55 title('Espacio de Fases 3D en Tiempo Real');
56
57 % Límites fijos para evitar que los ejes salten durante la animación
58 xlim([-pi, pi]);
59 ylim([-pi, pi]);
60 zlim([-10, 10]);
61
62 grid on;
63 view(45, 30); % Cambiamos la perspectiva de la cámara 3D
64 rotate3d on; % Permite hacer clic y rotar el gráfico mientras corre
65
66 % Opciones de precisión para el integrador
67 options = odeset('RelTol', 1e-6, 'AbsTol', 1e-8);
68
69 % --- BUCLE DE SIMULACIÓN EN DIRECTO ---
70 fprintf('Iniciando animación... Puedes rotar el gráfico con el ratón.\n');
71
72 for t_actual = 0:dt:t_total
73 % Si la ventana gráfica se cierra, detenemos el programa
74 if ~ishandle(h_trayectoria)
75 break;
76 end
77
78 % Resolver el sistema solo para el pequeño intervalo [t_actual, t_actual + dt]
79 [~, z_sol] = ode45(@(t, z) derivadas(t, z, g, l, m), [t_actual, t_actual + dt], z
, options);
80
81 % El último punto de esta solución será el estado actual para la siguiente
82 iteración

```

```

80     z = z_sol(end, :);
81
82     % Normalizar los ángulos al rango [-pi, pi] para que no se escapen del gráfico
83     theta1_norm = mod(z(1) + pi, 2*pi) - pi;
84     theta2_norm = mod(z(2) + pi, 2*pi) - pi;
85     omega2_actual = z(4);
86
87     % Añadir el nuevo punto a la línea animada
88     addpoints(h_trayectoria, theta1_norm, theta2_norm, omega2_actual);
89
90     % Actualizar la posición del punto rojo indicador
91     set(h_estado_actual, 'XData', theta1_norm, 'YData', theta2_norm, 'ZData',
        omega2_actual);
92
93     % Forzar a Matlab a dibujar los cambios en la pantalla
94     drawnow;
95
96     if guardar_video
97         frame = getframe(gcf);
98         writeVideo(v, frame);
99     end
100 end
101 fprintf('Animación finalizada.\n');
102
103 if guardar_video
104     close(v);
105     disp('Video guardado y exportado con éxito!');
106 end
107 end
108
109 % --- MOVIMIENTO DEL PÉNDULO DOBLE ---
110 function dzdt = derivadas(~, z, g, l, m)
111     t1 = z(1); t2 = z(2);
112     w1 = z(3); w2 = z(4);
113
114     % Matriz de inercia
115     M = [2*m*l^2,          m*l^2*cos(t1 - t2);
          m*l^2*cos(t1 - t2), m*l^2];
116
117     % Fuerzas centrífugas y gravedad
118     F = [-2*m*g*l*sin(t1) - m*l^2*w2^2*sin(t1 - t2);
          -m*g*l*sin(t2)   + m*l^2*w1^2*sin(t1 - t2)];
119
120     % Aceleraciones
121     alpha = M \ F;
122
123     dzdt = [w1; w2; alpha(1); alpha(2)];
124
125 end
126

```

F. Código 6: Secciones de Poincaré múltiples para distintos niveles de energía

```

1  function Codigo6_secciones_Poincare_multiples()
2  % Representación de la secciones de Poincaré múltiples para el péndulo
3  % doble. En cada subplot se muestran varias secciones de Poincaré
4  % superpuestas de distintas trayectorias (cada una de un mismo color) que
5  % tienen la misma energía.
6  close all;
7  % --- Configuración de parámetros físicos (n=2) ---
8  g = 9.81;
9  l1 = 0.5; l2 = 0.5;
10 m1 = 10.0; m2 = 10.0;
11
12 % --- REFERENCIA DE ENERGÍA ---
13 % E = 0 J en reposo.
14 E_deseada = [2 15 40 70 100 150];

```

```

15 num_trayectorias = 50;
16 tspan = [0, 150];
17
18
19 figure('Color', 'w', 'Position', [100, 100, 800, 600]);
20 hold on; grid on; box on;
21
22 fprintf('Calculando Sección de Poincaré para Energía = %.2f J', E_deseada);
23
24 % Rango de muestreo para theta1 (ajustado para energías bajas)
25 theta1_vals = linspace(-0.4, 0.4, num_trayectorias);
26
27 sk=0;
28 sgtitle('Secciones de Poincaré', 'FontSize', 20);
29
30 for E=E_deseada
31     sk=sk+1;
32     subplot(2,3,sk);
33     hold on;
34     title(sprintf('Energía = %.0f J', E), 'FontSize', 14);
35     xlabel('\theta_1 (rad)', 'FontSize', 12);
36     ylabel('\omega_1 (rad/s)', 'FontSize', 12);
37
38
39     for i = 1:num_trayectorias
40         th1 = theta1_vals(i);
41         th2 = 0;
42         om2 = 0;
43
44         % Energía Potencial MODIFICADA: Se le suma el desfase para que en (0,0) sea V = 0
45         V_original = -m1*g*l1*cos(th1) - m2*g*(l1*cos(th1) + l2*cos(th2));
46         V_offset = (m1 + m2)*g*l1 + m2*g*l2; % Energía potencial en reposo
47         V = V_original + V_offset;
48
49         % Energía Cinética requerida
50         T_req = E - V;
51
52         if T_req < 0
53             continue; % Energía inalcanzable para este ángulo inicial
54         end
55
56         % Velocidad angular inicial om1
57         om1 = sqrt(2 * T_req / ((m1 + m2) * l1^2));
58         z0 = [th1; th2; om1; om2];
59
60         % Configuración del detector de eventos (Sección de Poincaré)
61         opts = odeset('RelTol', 1e-7, 'AbsTol', 1e-9, 'Events', @event_poincare);
62
63         % Simulación
64         [~, ~, te, ye, ie] = ode45(@(t, z) ec_movimiento(z, m1, m2, l1, l2, g), tspan, z0, opts);
65
66         % Filtrar y graficar
67         if ~isempty(ye)
68             validos = ye(:,4) > 0; % Condición omega2 > 0
69             th1_puntos = ye(validos, 1);
70             om1_puntos = ye(validos, 3);
71
72             % Normalizar ángulo
73             th1_puntos = mod(th1_puntos + pi, 2*pi) - pi;
74             plot(th1_puntos, om1_puntos, '.', 'MarkerSize', 4);
75         end
76     end
77
78 end
79
80 end
81
82 %% --- FUNCIÓN DE EVENTOS ---
83 function [value, isterminal, direction] = event_poincare(~, z)
84     value = z(2); % Detectar theta2 = 0

```

```

85     isterminal = 0;
86     direction = 0;
87 end
88
89 %% --- ECUACIONES DE MOVIMIENTO ---
90 % Nota: Las ecuaciones de movimiento NO cambian, ya que las fuerzas dependen
91 % de las derivadas del potencial (gradientes), y la constante que sumamos desaparece al
    derivar.
92 function dzdt = ec_movimiento(z, m1, m2, l1, l2, g)
93     th1 = z(1); th2 = z(2);
94     om1 = z(3); om2 = z(4);
95
96     M = [ (m1 + m2)*l1^2,          m2*l1*l2*cos(th1 - th2);
97           m2*l1*l2*cos(th1 - th2), m2*l2^2 ];
98
99     F = [ -m2*l1*l2*om2^2*sin(th1 - th2) - (m1 + m2)*g*l1*sin(th1);
100           m2*l1*l2*om1^2*sin(th1 - th2) - m2*g*l2*sin(th2) ];
101
102     alpha = M \ F;
103     dzdt = [om1; om2; alpha(1); alpha(2)];
104 end

```